深度學習網路入侵偵測比較研究之重現與訓練穩定性分析

Reproducing "A Comparative Study of Using Deep Learning Algorithms in Network Intrusion Detection" and Analyzing Training Instability of Recurrent Models

張鈞崴

摘要—本報告重現 Elsayed 等人於 IEEE Access (2024) 發表的深度學習網路入侵偵測比較研究 [1]。原論文在 NSL-KDD 資料集 [2] 上比較 DNN、RNN、CNN、LSTM、GRU 與 Hybrid CNN-LSTM 六種模型的二元與五類分類表現，並報告所有模型準確率介於 97.96% 至 99.54%，其中 GRU 為二元分類最佳模型。我們依原論文公開的前處理流程與 KerasTuner 最佳超參數重新實作 12 個模型，在 25,195 筆測試資料上評估。結果顯示 DNN、CNN、RNN 在兩種任務皆成功重現（與原論文差距在 ±0.4 個百分點內），但 LSTM 與 GRU 的二元分類準確率僅 96.41% 與 96.06%，較原論文低約 3 個百分點，原論文「GRU 最佳」的結論未能重現，本次最佳模型為 RNN（99.47%）。為找出差異原因，我們設計五項診斷實驗。結果顯示主因是循環模型的訓練不穩定：僅更換隨機種子，準確率即變動 1.4 至 4.5 個百分點；加入梯度裁剪後所有訓練崩落現象消失，GRU 準確率由 96.06% 提升至 98.78%；梯度累積並非原因。改用原論文的 TensorFlow 2.14 訓練 LSTM 得到 98.21%，與 PyTorch 未崩落的訓練相當，框架差異亦非主因。此外，我們指出原論文評估方式的潛在問題：測試集取自訓練檔案內的隨機切分，且將無時間順序的表格特徵視為序列輸入循環網路，使得 99% 以上的準確率可能高估模型對未知攻擊的偵測能力。

關鍵詞—網路入侵偵測、深度學習、NSL-KDD、可重現性、循環神經網路、訓練穩定性、梯度裁剪

I. 緒論

隨著網路服務與物聯網裝置快速普及，網路攻擊的數量與複雜度持續上升，入侵偵測系統（Intrusion Detection System, IDS）成為防禦體系的重要一環。傳統以特徵碼（signature）為基礎的偵測方法誤報率低，但只能辨識已知攻擊；以異常（anomaly）為基礎的方法則嘗試學習正常流量的行為模式，以偵測未知攻擊。近年深度學習因具備自動擷取特徵的能力，被大量應用於網路入侵偵測（NIDS）研究。

Elsayed 等人 [1] 在 NSL-KDD 資料集上系統性地比較六種深度學習架構，並以 KerasTuner 搜尋各模型的超參數，報告所有模型在二元與五類分類皆達到約 98%–99.5% 的準確率。由於該研究提供了完整的超參數設定，非常適合作為重現研究的對象。機器學習研究的可重現性近年受到高度重視 [11], [12]，單次實驗的數值往往受到隨機種子、框架版本與硬體差異影響，若沒有重現，很難判斷論文中模型間零點幾個百分點的差距是否具有意義。

本報告的目標有三：（1）依原論文設定完整重現 12 個模型（6 種架構 × 2 種任務）；（2）針對重現失敗的模型設計診斷實驗，找出差異的原因；（3）從資料與評估方式的角度，討論原論文結論的適用範圍。本報告的主要貢獻如下：

- 提供可直接執行的完整重現程式，涵蓋前處理、12 個模型、評估與圖表產生。
- 證實 DNN、CNN、RNN 與原論文結果一致，但 LSTM、GRU 與多元分類 CNN-LSTM 無法穩定重現。
- 透過換種子、梯度裁剪、調整學習率、移除梯度累積及改用原論文框架等診斷實驗，釐清循環模型結果偏低的主因。
- 指出原論文在評估流程與輸入表示上的潛在問題，並提出改進建議。

II. 原論文方法概述

A. 資料集

NSL-KDD [2] 是 KDD Cup 99 的改良版本，移除了大量重複紀錄，是入侵偵測領域最常用的基準資料集之一。每筆紀錄描述一條網路連線，包含 41 個特徵（38 個數值、3 個類別）與一個標籤。攻擊被歸為四大類：阻斷服務（DoS）、探測（Probe）、遠端到本地（R2L）與使用者提權（U2R）。

B. 前處理與模型

原論文對數值特徵做標準化，對 protocol_type、service、flag 三個類別特徵做 one-hot 編碼，使特徵維度由 41 增為 122，接著以 80/20 隨機切分訓練與測試資料，再從訓練資料取 20% 作驗證。六種模型皆以 KerasTuner 隨機搜尋決定單元數、卷積核數量與大小、dropout 比例等超參數，最後以 Adam 最佳化器、100 epochs、batch size 5000 統一訓練。

C. 原論文主要結論

- 二元分類以 GRU 最佳（準確率 99.54%），LSTM、CNN、DNN 次之。
- 五類分類以 DNN 表現最突出（99.34%），Hybrid CNN-LSTM 最低（97.96%）。

• 所有模型在 DoS、Probe 與 Normal 類別接近完美，但 U2R 對所有模型都是挑戰。

III. 重現方法

A. 資料與前處理

我們使用 KDDTrain+ 的全部 125,973 筆資料，依原論文以 StandardScaler 標準化 38 個數值特徵，one-hot 編碼後得到 122 維特徵。依攻擊名稱對照表將訓練資料中的 22 種攻擊歸入四大類。資料以隨機種子 42 切分為 100,778 筆訓練與 25,195 筆測試資料，訓練時再以最後 20% 作為驗證集。圖 1 為整體流程，圖 2 為各類別樣本數。

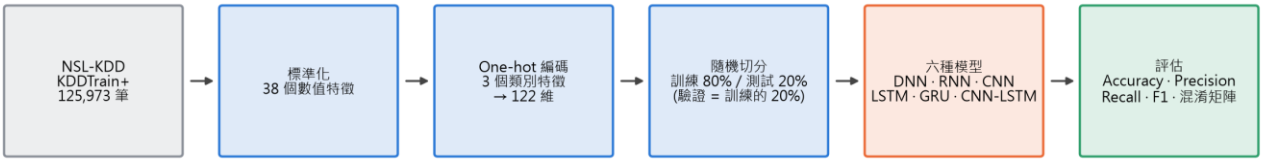


圖 1. 重現實驗的整體流程。

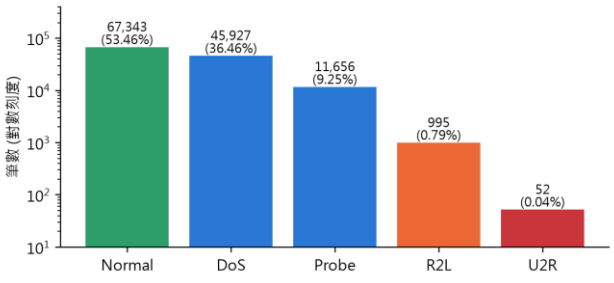


圖 2. KDDTrain+ 各類別樣本數（對數刻度）。U2R 只有 52 筆，佔 0.04%，類別極度不平衡。

如圖 2 所示，資料集類別極度不平衡：Normal 與 DoS 合計佔 89.9%，而 U2R 只有 52 筆。隨機切分後，測試集中 U2R 僅剩 11 筆、R2L 224 筆，這兩類的指標將有很大的統計波動。

B. 模型架構

所有超參數直接採用原論文第 VI 節報告的 KerasTuner 最佳值，未重新搜尋。序列型模型（RNN、CNN、LSTM、GRU、CNN-LSTM）將 122 維特徵視為長度 122、每步 1 維的序列。表 1 列出二元分類模型的結構，五類分類的結構見附錄表 A1。

表 1. 二元分類模型結構（原論文最佳超參數）

模型	結構（二元分類）
DNN	Dense 32 → Dense 32
RNN	SimpleRNN 50/32/8 (dropout 0.3/0.1) → Dense 16
CNN	Conv1D 64(k=7) → Pool 4 → Conv1D 32(k=7) → Pool 2 → Dense 50
LSTM	LSTM 50/16/16 (dropout 0.2/0.1) → Dense 32
GRU	GRU 128/128/16 (dropout 0.2/0.4) → Dense 25
CNN-LSTM	Conv1D 64(k=7) → Pool 2 → LSTM 128 (dropout 0.2)

原論文的程式碼以圖片呈現，部分細節無法確認，我們做了以下推測：CNN 第二個卷積層之後的池化大小為 2；Hybrid CNN-LSTM 卷積層之後的池化大小為 2；所有循環模型的 Dense 層使用 ReLU 活化函數。

C. 訓練與評估

二元分類輸出層使用 sigmoid 搭配 binary cross-entropy；五類分類使用 softmax 搭配 categorical cross-entropy。最佳化器為 Adam [5]（學習率 10^{-3}），訓練 100 epochs，batch size 5000。評估指標與原論文相同：

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (1)$$

$$Precision = TP / (TP + FP) \quad (2)$$

$$Recall = TP / (TP + FN) \quad (3)$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall) \quad (4)$$

由原論文的混淆矩陣可推知其二元分類以 normal 為正類，我們採用相同定義。五類分類的 Precision 與 Recall 依 Keras 指標的算法，以 one-hot 標籤、門檻 0.5 做 micro 平均，因原論文報告的 recall 低於 accuracy，與此算法相符。

D. 實驗環境與差異

表 2. 原論文與本重現的實驗環境比較

項目	原論文	本重現
框架	TF / Keras 2.14	Keras 3 + PyTorch 2.6
硬體	Colab TPU	RTX 4050 (6 GB)
Batch	5000	5000（循環模型 1000×5 累積）
評估權重	未說明	最後一個 epoch
重複次數	未說明	1 次（種子 42）

RTX 4050 的 6 GB 記憶體無法容納 batch 5000 的 GRU 訓練，因此 RNN、LSTM、GRU 與 CNN-LSTM 改為每次輸入 1000 筆、累積 5 次梯度才更新權重。數學上這與 batch 5000 等效，但框架實作細節可能不同，第 V 節將以實驗檢驗此差異的影響。

IV. 重現結果

A. 二元分類

表 3 與圖 3 比較本重現與原論文的二元分類結果。

表 3. 二元分類結果（本重現 / 原論文，單位 %）

模型	Accuracy	Precision	Recall	F1	Accuracy 差距
DNN	99.43 / 99.39	99.47 / 99.45	99.46 / 99.40	99.46 / 99.43	+0.04
RNN	99.47 / 99.09	99.32 / 99.09	99.68 / 99.21	99.50 / 99.15	+0.37
CNN	99.40 / 99.38	99.35 / 99.34	99.52 / 99.49	99.44 / 99.42	+0.02
LSTM	96.41 / 99.41	96.74 / 99.46	96.51 / 99.43	96.62 / 99.44	**-3.00**
GRU	96.06 / 99.54	95.93 / 99.46	96.71 / 99.67	96.32 / 99.56	**-3.48**
CNN-LSTM	99.16 / 98.94	99.18 / 98.94	99.24 / 99.08	99.21 / 99.01	+0.21

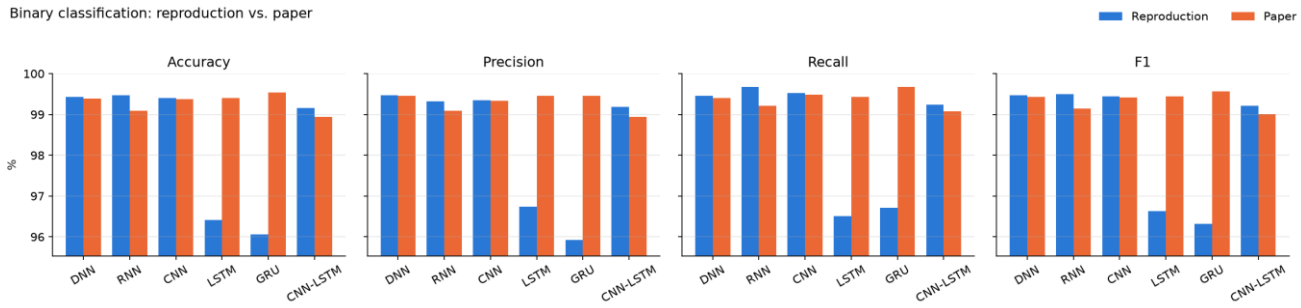


圖 3. 二元分類各指標比較。藍色為本重現，橘色為原論文。LSTM 與 GRU 明顯低於原論文，其餘模型一致。

表 3 顯示，DNN、CNN、RNN 與 CNN-LSTM 的準確率均與原論文相差 0.4 個百分點以內，RNN（99.47%）甚至比原論文高 0.37 個百分點，成為本次重現中二元分類最佳的模型。相對地，LSTM 與 GRU 只有 96.41% 與 96.06%，分別低於原論文 3.00 與 3.48 個百分點。原論文宣稱 GRU 為最佳模型，在本次重現中 GRU 反而是最差的。

表 4. 二元分類混淆矩陣（正類為 normal）

模型	TN	FP	FN	TP
DNN	11,702	71	73	13,349
RNN	11,682	91	43	13,379
CNN	11,686	87	64	13,358
LSTM	11,336	437	468	12,954
GRU	11,222	551	442	12,980
CNN-LSTM	11,663	110	102	13,320

表 4 的混淆矩陣顯示，表現良好的模型誤判數皆在 100 筆上下，原論文報告的 DNN 為 74 FP / 80 FN，與本重現的 71 / 73 非常接近。LSTM 與 GRU 的誤判數則多出四到六倍。

B. 五類分類

表 5 為五類分類的整體結果，表 6 為各類別的 F1-score。

表 5. 五類分類結果（本重現 / 原論文，單位 %）

模型	Accuracy	Precision	Recall	F1	Accuracy 差距
DNN	99.43 / 99.34	99.45 / 99.36	99.42 / 99.33	99.44 / 99.34	+0.09
RNN	99.42 / 99.25	99.43 / 99.25	99.39 / 99.24	99.41 / 99.25	+0.17
CNN	99.41 / 99.39	99.41 / 99.40	99.41 / 99.37	99.41 / 99.39	+0.02
LSTM	98.05 / 99.39	98.12 / 99.41	98.01 / 99.39	98.07 / 99.40	**-1.34**
GRU	98.78 / 99.38	98.84 / 99.39	98.71 / 99.37	98.77 / 99.38	-0.60
CNN-LSTM	93.42 / 97.96	94.53 / 98.01	92.75 / 97.79	93.63 / 97.90	**-4.54**

表 5 中，DNN、RNN、CNN 再次成功重現，三者準確率介於 99.41% 至 99.43%，彼此差距僅 0.02 個百分點，單次實驗無法分出優劣。原論文「DNN 最佳」的結論在數值上成立，但差距不具統計意義。LSTM、GRU 再度偏低，CNN-LSTM 則只有 93.42%，比原論文低 4.54 個百分點。

表 6. 五類分類各類別 F1-score

模型	DoS (n=9,181)	Normal (n=13,422)	Probe (n=2,357)	R2L (n=224)	U2R (n=11)
DNN	0.998	0.995	0.989	0.876	0.167
RNN	0.998	0.995	0.988	0.872	0.308
CNN	0.999	0.995	0.990	0.841	0.375
LSTM	0.991	0.983	0.951	0.674	0.000
GRU	0.997	0.989	0.970	0.686	0.000
CNN-LSTM	0.963	0.941	0.833	0.009	0.000

表 6 的各類別結果與原論文的描述一致：DoS、Normal、Probe 三類的 F1 接近 1，DNN、RNN、CNN 在 R2L 上達到 0.84–0.88，而 U2R 對所有模型都極為困難，最好的 CNN 也只有 0.375，LSTM、GRU 與 CNN-LSTM 則完全無法偵測。值得注意的是，整體準確率高

達 99% 主要來自 DoS 與 Normal 兩個大類，少數類別的失敗幾乎不影響整體準確率，因此單看準確率會高估模型的實際偵測能力。

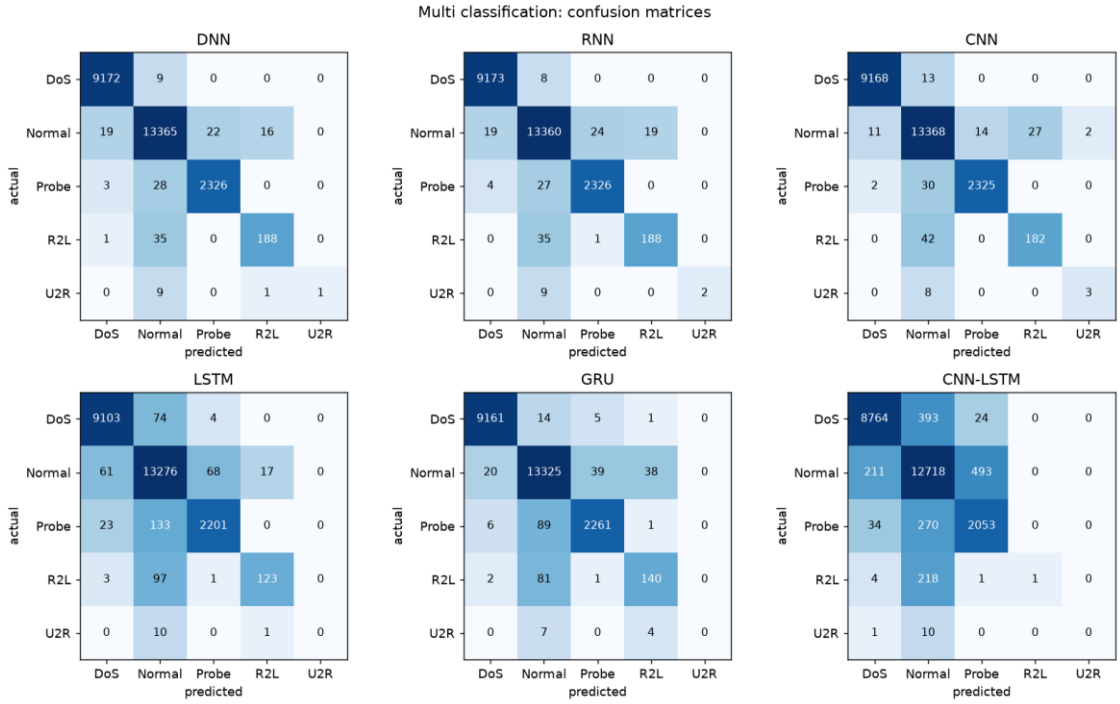


圖 4. 五類分類混淆矩陣（色階為對數刻度）。U2R 與 R2L 大量被誤判為 Normal。

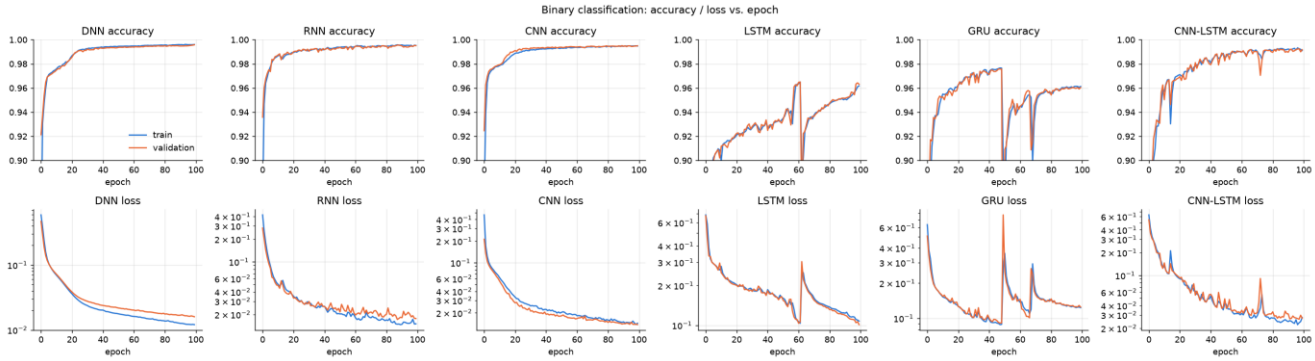


圖 5. 二元分類訓練與驗證曲線。DNN、RNN、CNN 平滑收斂；LSTM 與 GRU 在訓練中途出現準確率驟降與損失暴增。

C. 訓練曲線觀察

圖 5 揭示了 LSTM、GRU 結果偏低的線索。首先，這兩個模型的訓練與驗證準確率幾乎重疊，代表問題**不是過擬合**，而是模型沒有訓練好。其次，曲線出現明顯的「崩落」：二元 GRU 在約第 48 epoch 準確率由 97.6% 驟降，損失同時暴增，之後雖逐漸回升，但到第 100 epoch 仍未回到崩落前的水準；二元 LSTM 在約第 60 epoch 也有類似現象。這種現象是循環網路訓練中典型的梯度暴衝 [6], [7]。根據這些觀察，我們在下一節設計診斷實驗。

V. 差異原因之診斷實驗

A. 假設與實驗設計

我們針對重現失敗最明顯的三個設定（二元 LSTM、二元 GRU、五類 CNN-LSTM），提出五個可能造成差異的假設，並為每個假設設計對應實驗（表 7）。每個實驗同時記錄「最後一個 epoch」與「驗證準確率最高 epoch」兩組權重的測試準確率，以區分「訓練不穩定」與「模型能力不足」。

表 7. 診斷實驗設計

假設	對應實驗
H1 隨機種子造成的變異	換用種子 1 重新訓練
H2 梯度暴衝造成崩落	加入梯度裁剪 clipnorm = 1 [6]
H3 學習率過大	學習率降為 3×10^{-4}
H4 梯度累積實作差異	LSTM 直接用 batch 5000 訓練
H5 框架差異	以 TensorFlow 2.14 訓練 LSTM

B. 實驗結果

表 8 為所有診斷實驗的結果，圖 6 為對應的驗證準確率曲線。

表 8. 診斷實驗結果（測試準確率 %）。崩落次數為驗證準確率比先前最高值低 5 個百分點以上的 epoch 數

設定	實驗	最後 epoch 權重	最佳 epoch 權重	最佳 epoch	崩落次數
二元 LSTM	主實驗 (種子 42)	96.41	—	62	1
二元 LSTM	換種子 (種子 1)	97.81	98.04	97	0
二元 LSTM	梯度裁剪 clipnorm=1	97.96	97.96	100	0
二元 LSTM	學習率 3e-4	93.24	93.31	99	0
二元 LSTM	不做梯度累積	97.72	97.72	100	0
二元 LSTM	TensorFlow 2.14	98.21	98.19	93	0
二元 GRU	主實驗 (種子 42)	96.06	—	47	5
二元 GRU	換種子 (種子 1)	98.02	98.34	89	1
二元 GRU	梯度裁剪 clipnorm=1	98.78	98.79	97	0
二元 GRU	學習率 3e-4	97.33	97.33	100	0
五類 CNN-LSTM	主實驗 (種子 42)	93.42	—	96	7
五類 CNN-LSTM	換種子 (種子 1)	97.96	98.40	99	6
五類 CNN-LSTM	梯度裁剪 clipnorm=1	97.29	97.82	97	0
五類 CNN-LSTM	學習率 3e-4	96.04	96.04	100	1

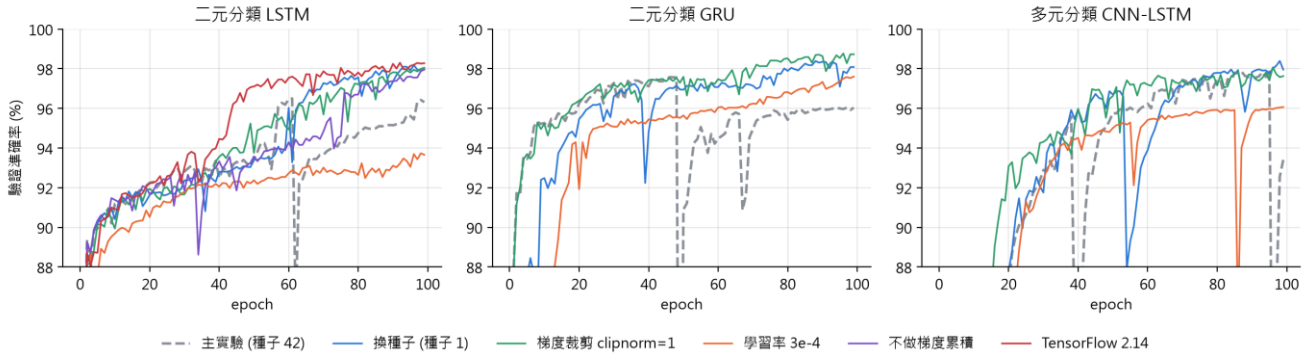


圖 6. 診斷實驗的驗證準確率曲線。灰色虛線為主實驗；加入梯度裁剪（綠）後崩落消失，換種子（藍）的結果明顯較好，降低學習率（橘）則收斂明顯變慢。

H1 隨機種子：只把種子由 42 換成 1，二元 LSTM 由 96.41% 升至 97.81%，二元 GRU 由 96.06% 升至 98.02%，五類 CNN-LSTM 更由 93.42% 升至 97.96%，恰好等於原論文報告的 97.96%。同一設定僅因種子不同就相差 1.4–4.5 個百分點，大於原論文中六個模型之間不到 1 個百分點的差距。

H2 梯度暴衝：加入梯度裁剪（clipnorm = 1）後，三個設定的崩落次數全部降為 0，「最後 epoch」與「最佳 epoch」的準確率幾乎相同，訓練變得穩定。二元 GRU 達到 98.78%，是所有診斷實驗中最高的結果；二元 LSTM 為 97.96%，五類 CNN-LSTM 為 97.29%（最佳 epoch 為 97.82%）。這證實主實驗中的崩落是梯度暴衝所致 [6]。

H3 學習率：把學習率降為 3×10^{-4} 反而使結果變差：二元 LSTM 只有 93.24%，二元 GRU 97.33%，五類 CNN-LSTM 96.04%，且最佳 epoch 都落在第 99–100 epoch，代表模型仍在進步、尚未收斂。在 batch 5000、100 epochs（約 1,600 次權重更新）的預算下，降低學習率只會讓收斂更慢。

H4 梯度累積：二元 LSTM 不做梯度累積、直接以 batch 5000 訓練，準確率為 97.72%，沒有崩落，與換種子（97.81%）及梯度裁剪（97.96%）的結果相當。因此梯度累積不是造成差異的原因；主實驗的 96.41% 是因為第 60 epoch 附近發生一次崩落，屬於隨機事件。

H5 框架差異：以原論文使用的 TensorFlow 2.14 直接以 batch 5000 訓練二元 LSTM，準確率為 98.21%，崩落 0 次。此結果高於 PyTorch 主實驗（96.41%，曾崩落），但與 PyTorch 未崩落的各組（換種子 97.81%、梯度裁剪 97.96%、不做梯度累積 97.72%）相當，且仍低於原論文的 99.41%。換言之，TensorFlow 版本的優勢來自這次訓練沒有發生崩落，而非框架本身；框架差異無法解釋與原論文的落差。

C. 小結

綜合以上實驗，LSTM、GRU 與 CNN-LSTM 無法重現的主因是**訓練不穩定**：循環網路在學習率 10^{-3} 下會間歇發生梯度暴衝，使準確率突然崩落，而 100 epochs 的訓練預算不足以讓模型從崩落中完全恢復。崩落何時發生取決於隨機種子，因此同一模型的結果會有數個百分點的落差。梯度累積不是主因；框架差異也不是主因。

然而，即使加入梯度裁剪使訓練穩定，二元 LSTM 與 GRU 的最佳結果（97.96% 與 98.78%）仍比原論文低 0.8–1.5 個百分點。LSTM 的驗證曲線到第 100 epoch 仍在上升，顯示模型在此預算下尚未收斂。原論文的數值可能來自較幸運的種子，或是 KerasTuner 從大量試驗中

挑選最佳結果所產生的選擇效應；由於原論文沒有報告種子與重複次數，我們無法進一步確認。

這個發現對原論文的模型排名有直接影響：原論文以 0.1–0.2 個百分點的差距判定 GRU 為最佳模型，但本研究顯示單一模型因種子不同就可能有超過 4 個百分點的差異，因此在沒有重複實驗的情況下，這樣的排名並不可靠。

VI. 討論

A. 模型複雜度與效益

表 9. 二元分類模型的參數量、訓練時間與準確率

模型	參數量	訓練時間 (s)	準確率 (%)
DNN	5,025	36	99.43
RNN	5,745	4388	99.47
CNN	32,581	72	99.40
LSTM	17,377	252	96.41
GRU	156,835	584	96.06
CNN-LSTM	99,457	281	99.16

表 9 顯示，最簡單的 DNN 只有 5,025 個參數、訓練 36 秒，準確率 99.43% 已與最佳模型相差無幾；RNN 雖然準確率最高，訓練卻需要 4388 秒（約 73 分鐘），是 DNN 的 122 倍。GRU 參數量是 DNN 的 31 倍，表現反而最差。對 NSL-KDD 這類表格資料而言，增加模型複雜度並未帶來可觀的效益。

B. 將表格特徵視為序列的合理性

原論文主張循環網路能捕捉網路流量的「時間依賴」。然而 NSL-KDD 的每筆紀錄是一條獨立連線的統計摘要，122 個特徵是由數值欄位與 one-hot 欄位依固定順序排列而成，彼此之間並沒有時間先後關係。把它們當成長度 122 的序列餵給 RNN，模型學到的只是欄位排列順序上的人為結構，而非流量的時間行為。本重現中，不具序列假設的 DNN 已達到與最佳模型相同的水準，而依賴序列假設的 LSTM、GRU 反而最不穩定，這支持了上述觀點。若要真正利用時間資訊，應以同一主機或同一時間窗內的連續連線作為序列。

C. 評估方式的限制

原論文與本重現都只使用 KDDTrain+ 內部的隨機切分。官方的 KDDTest+ 含有 17 種訓練集未出現的攻擊類型 [2]，更能反映模型面對新型攻擊的能力，在 KDDTest+ 上評估的準確率通常明顯低於隨機切分的結果。因此 99% 以上的數字應解讀為「對已知攻擊類型的辨識能力」，而非對未知攻擊的泛化能力。此外，隨機切分可能讓高度相似的連線同時出現在訓練與測試資料中，進一步推高準確率。Arp 等人 [13] 將這類評估偏差列為資安機器學習的常見陷阱。

D. 類別不平衡

U2R 在測試集只有 11 筆，即使模型全部判錯，整體準確率也只下降 0.04 個百分點。若入侵偵測系統的重點在於找出少見但危害大的攻擊，應改以各類別的 recall、macro-F1 等指標評估，並搭配 SMOTE [14] 或類別權重等方法處理不平衡問題。

VII. 結論

本報告完整重現了 Elsayed 等人 [1] 的 12 個深度學習入侵偵測模型。DNN、CNN 與 RNN 在二元與五類分類皆成功重現，與原論文差距在 0.4 個百分點以內；LSTM、GRU 與五類 CNN-LSTM 則明顯偏低，原論文「GRU 為最佳二元分類模型」的結論未能重現。

診斷實驗指出差異來自循環模型的訓練不穩定，而非實作錯誤：換種子即可讓準確率變動 4.5 個百分點，加入梯度裁剪可消除崩落並使 GRU 提升至 98.78%。這說明在比較模型時，必須以多個種子重複實驗並報告變異，否則零點幾個百分點的差距不具意義。

此外，最簡單的 DNN 只需 36 秒訓練即可達到 99.43%，與最複雜的模型相當，顯示在 NSL-KDD 這類表格資料上，循環與卷積結構並未帶來實質優勢。而原論文採用的隨機切分評估方式，也使 99% 以上的準確率可能高估模型對未知攻擊的偵測能力。

未來工作包括：以多個隨機種子重複實驗並報告平均與標準差；在官方 KDDTest+ 上評估模型對未知攻擊的能力；針對少數類別加入資料平衡方法；以及在具備真實時間序列結構的資料集上重新檢驗循環網路的優勢。

參考文獻

[1] S. Elsayed, K. Mohamed, and M. A. Madkour, “A comparative study of using deep learning algorithms in network intrusion detection,” IEEE Access, vol. 12, pp. 58851–58870, 2024, doi: 10.1109/ACCESS.2024.3389096.

[2] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl. (CISDA), 2009, pp. 1–6.

[3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Cho et al., “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in Proc. EMNLP, 2014, pp. 1724–1734.

[5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. ICLR, 2015.

[6] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in Proc. ICML, 2013, pp. 1310–1318.

[7] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” IEEE Trans. Neural Netw., vol. 5, no. 2, pp. 157–166, 1994.

[8] F. Chollet et al., “Keras,” 2015. [Online]. Available: https://keras.io

[9] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Proc. NeurIPS, 2019, pp. 8024–8035.

[10] T. O’Malley et al., “KerasTuner,” 2019. [Online]. Available: https://github.com/keras-team/keras-tuner

[11] X. Bouthillier, C. Laurent, and P. Vincent, “Unreproducible research is reproducible,” in Proc. ICML, 2019, pp. 725–734.

[12] J. Pineau et al., “Improving reproducibility in machine learning research,” J. Mach. Learn. Res., vol. 22, no. 164, pp. 1–20, 2021.

[13] D. Arp et al., “Dos and don’ts of machine learning in computer security,” in Proc. USENIX Security Symp., 2022, pp. 3971–3988.

[14] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.

附錄

A. 五類分類模型結構

表 A1. 五類分類模型結構

模型	結構（五類分類）
DNN	Dense 64 → Dense 8
RNN	SimpleRNN 64/32/64 (dropout 0.1/0.2) → Dense 25
CNN	Conv1D 64(k=7) → Pool 4 → Conv1D 32(k=7) → Pool 2 → Dense 50
LSTM	LSTM 64/32/50 (dropout 0.1/0.5) → Dense 64
GRU	GRU 128/50/25 (dropout 0.2/0.1) → Dense 16
CNN-LSTM	Conv1D 8(k=5) → Pool 2 → LSTM 64 (dropout 0.2)

B. 重現步驟

所有程式與結果皆置於專案資料夾。nids_repro.py 負責前處理、模型定義、訓練與評估；make_report.py 產生對照表與圖；experiments.py 與 tf_check.py 為第 V 節的診斷實驗；build_report.py 產生本報告。在已安裝套件的虛擬環境中依序執行：

```
python nids_repro.py
python make_report.py
python experiments.py
python report_figures.py
python build_report.py
```